

Cognizant-upgrad Training Program

Uge_dotnet_FSD_Python

Trainee Information

Name: *Golla Amulya*

Email: *gollaamulya418@gmail.com*

Batch Name: *B1_Python*

Course: *uge_dotnet FSD (Python)*

Training Duration: *16th Feb 2026 – 26th May 2026*

Session Timings: *9:30 AM – 6:30 PM*

Faculty: *Kunal*

Week 6

PROJECT 1:- SMART SKILL EXCHANGE

Booking.cs

using System.ComponentModel.DataAnnotations;

namespace SmartSkillExchange.Models

{

public class Booking

{

public int Id { get; set; }

[Required]

public int SkillId { get; set; }

[Required]

public int UserId { get; set; }

public DateTime BookedAt { get; set; } = DateTime.Now;

public string Status { get; set; } = "Pending";

public Skill? Skill { get; set; }

public User? User { get; set; }

}

}

Review.cs

using System.ComponentModel.DataAnnotations;

namespace SmartSkillExchange.Models

```
{
    public class Review
    {
        public int Id { get; set; }

        [Required]
        [StringLength(500)]
        public string Comment { get; set; } = string.Empty;

        [Range(1, 5)]
        public int Rating { get; set; }

        [Required]
        public int SkillId { get; set; }

        public int UserId { get; set; }

        public Skill? Skill { get; set; }
        public User? User { get; set; }
    }
}
```

Skill.cs

using System.ComponentModel.DataAnnotations;

namespace SmartSkillExchange.Models

```
{
    public class Skill
    {
        public int Id { get; set; }

        [Required]
        [StringLength(100)]
        public string Title { get; set; } = string.Empty;

        [Required]
        [StringLength(500)]
        public string Description { get; set; } = string.Empty;
    }
}
```

```
[Required]
public string Category { get; set; } = string.Empty;
```

```
[Required]
[Range(1, 500)]
public int PointCost { get; set; }
```

```
public int UserId { get; set; }
```

```
public User? User { get; set; }
```

```
public List<Booking> Bookings { get; set; } = new List<Booking>();
public List<Review> Reviews { get; set; } = new List<Review>();
```

```
}
```

```
}
```

User.cs

```
using System.ComponentModel.DataAnnotations;
```

```
namespace SmartSkillExchange.Models
```

```
{
```

```
    public class User
```

```
    {
```

```
        public int Id { get; set; }
```

```
[Required]
```

```
[StringLength(100)]
```

```
public string Name { get; set; } = string.Empty;
```

```
[Required]
```

```
[EmailAddress]
```

```
public string Email { get; set; } = string.Empty;
```

```
[Required]
```

```
public string Password { get; set; } = string.Empty;
```

```
public List<Skill> Skills { get; set; } = new List<Skill>();
```

```
public List<Booking> Bookings { get; set; } = new List<Booking>();
```

```
}
```

```
}  
Wallet.cs  
using System.ComponentModel.DataAnnotations;
```

```
namespace SmartSkillExchange.Models
```

```
{  
    public class Wallet  
    {  
        public int Id { get; set; }  
  
        public int Points { get; set; } = 100;  
  
        [Required]  
        public int UserId { get; set; }  
  
        public User? User { get; set; }  
    }  
}
```

```
AppDbContext.cs
```

```
using Microsoft.EntityFrameworkCore;  
using SmartSkillExchange.Models;
```

```
namespace SmartSkillExchange.Data
```

```
{  
    public class AppDbContext : DbContext  
    {  
        public AppDbContext(DbContextOptions<AppDbContext> options)  
            : base(options)  
        {  
        }  
  
        public DbSet<User> Users { get; set; }  
        public DbSet<Skill> Skills { get; set; }  
        public DbSet<Booking> Bookings { get; set; }  
        public DbSet<Review> Reviews { get; set; }  
        public DbSet<Wallet> Wallets { get; set; }  
  
        protected override void OnModelCreating(ModelBuilder modelBuilder)  
        {  
            base.OnModelCreating(modelBuilder);  
        }  
    }  
}
```

```
// User → Skills
modelBuilder.Entity<Skill>()
    .HasOne(s => s.User)
    .WithMany(u => u.Skills)
    .HasForeignKey(s => s.UserId)
    .OnDelete(DeleteBehavior.NoAction);

// Booking → Skill
modelBuilder.Entity<Booking>()
    .HasOne(b => b.Skill)
    .WithMany(s => s.Bookings)
    .HasForeignKey(b => b.SkillId)
    .OnDelete(DeleteBehavior.NoAction);

// Booking → User
modelBuilder.Entity<Booking>()
    .HasOne(b => b.User)
    .WithMany(u => u.Bookings)
    .HasForeignKey(b => b.UserId)
    .OnDelete(DeleteBehavior.NoAction);

// Review → Skill
modelBuilder.Entity<Review>()
    .HasOne(r => r.Skill)
    .WithMany(s => s.Reviews)
    .HasForeignKey(r => r.SkillId)
    .OnDelete(DeleteBehavior.NoAction);

// Review → User
modelBuilder.Entity<Review>()
    .HasOne(r => r.User)
    .WithMany()
    .HasForeignKey(r => r.UserId)
    .OnDelete(DeleteBehavior.NoAction);

// Wallet → User
modelBuilder.Entity<Wallet>()
    .HasOne(w => w.User)
    .WithOne()
    .HasForeignKey<Wallet>(w => w.UserId)
    .OnDelete(DeleteBehavior.NoAction);
```

```
    }  
  }  
}
```

AccountController.cs

```
using Microsoft.AspNetCore.Mvc;  
using Microsoft.EntityFrameworkCore;  
using SmartSkillExchange.Data;  
using SmartSkillExchange.Models;
```

```
namespace SmartSkillExchange.Controllers
```

```
{  
    public class AccountController : Controller  
    {  
        private readonly AppDbContext _context;  
  
        public AccountController(AppDbContext context)  
        {  
            _context = context;  
        }  
  
        public IActionResult Register()  
        {  
            return View();  
        }  
  
        [HttpPost]  
        [ValidateAntiForgeryToken]  
        public async Task<IActionResult> Register(User model)  
        {  
            if (!ModelState.IsValid)  
                return View(model);  
  
            bool exists = await _context.Users  
                .AnyAsync(u => u.Email == model.Email);  
  
            if (exists)  
            {  
                ModelState.AddModelError("Email", "Email already registered.");  
            }  
        }  
    }  
}
```

```
    return View(model);  
}
```

```
_context.Users.Add(model);  
await _context.SaveChangesAsync();
```

```
var wallet = new Wallet  
{  
    UserId = model.Id,  
    Points = 100  
};  
_context.Wallets.Add(wallet);  
await _context.SaveChangesAsync();
```

```
HttpContext.Session.SetInt32("UserId", model.Id);  
HttpContext.Session.SetString("UserName", model.Name);
```

```
TempData["Success"] = "Registration successful! You have 100 points.";  
return RedirectToAction("Index", "Skill");  
}
```

```
public IActionResult Login()  
{  
    return View();  
}
```

```
[HttpPost]  
[ValidateAntiForgeryToken]  
public async Task<IActionResult> Login(string email, string password)  
{  
    var user = await _context.Users  
        .FirstOrDefaultAsync(u => u.Email == email && u.Password == password);  
  
    if (user == null)  
    {  
        ModelState.AddModelError("", "Invalid email or password.");  
        return View();  
    }  
}
```

```

HttpContext.Session.SetInt32("UserId", user.Id);
HttpContext.Session.SetString("UserName", user.Name);

TempData["Success"] = "Welcome back, " + user.Name + "!";
return RedirectToAction("Index", "Skill");
}

```

```

[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Logout()
{
    HttpContext.Session.Clear();
    TempData["Success"] = "Logged out successfully.";
    return RedirectToAction("Login");
}
}
}

```

AdminController.cs

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using SmartSkillExchange.Data;

```

```

namespace SmartSkillExchange.Controllers
{
    public class AdminController : Controller
    {
        private readonly AppDbContext _context;

        public AdminController(AppDbContext context)
        {
            _context = context;
        }

        public async Task<IActionResult> Index()
        {
            var userId = HttpContext.Session.GetInt32("UserId");
            if (userId == null)
                return RedirectToAction("Login", "Account");
        }
    }
}

```



```
var users = await _context.Users  
    .ToListAsync();
```

```
ViewBag.TotalUsers = users.Count;  
ViewBag.TotalSkills = await _context.Skills.CountAsync();  
ViewBag.TotalBookings = await _context.Bookings.CountAsync();
```

```
return View(users);  
}
```

```
public async Task<IActionResult> AllBookings()  
{  
    var userId = HttpContext.Session.GetInt32("UserId");  
    if (userId == null)  
        return RedirectToAction("Login", "Account");
```

```
    var bookings = await _context.Bookings  
        .Include(b => b.Skill)  
        .Include(b => b.User)  
        .OrderByDescending(b => b.BookedAt)  
        .ToListAsync();
```

```
    return View(bookings);
```

```
}
```

```
}
```

```
}
```

BookingController.cs

```
using Microsoft.AspNetCore.Mvc;  
using Microsoft.EntityFrameworkCore;  
using SmartSkillExchange.Data;  
using SmartSkillExchange.Models;
```

```
namespace SmartSkillExchange.Controllers
```

```
{
```

```
    public class BookingController : Controller
```

```
    {
```

```
        private readonly ApplicationDbContext _context;
```

```
public BookingController(AppDbContext context)
{
    _context = context;
}
```

```
// — My Bookings (as Learner) —————
public async Task<ActionResult> Index()
{
    var userId = HttpContext.Session.GetInt32("UserId");
    if (userId == null)
        return RedirectToAction("Login", "Account");

    var bookings = await _context.Bookings
        .Include(b => b.Skill)
        .Include(b => b.User)
        .Where(b => b.UserId == userId)
        .ToListAsync();

    return View(bookings);
}
```

```
// — Teaching Sessions (as Teacher) —————
public async Task<ActionResult> TeachingSessions()
{
    var userId = HttpContext.Session.GetInt32("UserId");
    if (userId == null)
        return RedirectToAction("Login", "Account");

    // All bookings where the SKILL belongs to the logged-in user
    var bookings = await _context.Bookings
        .Include(b => b.Skill)
        .Include(b => b.User)
        .Where(b => b.Skill!.UserId == userId)
        .OrderByDescending(b => b.BookedAt)
        .ToListAsync();

    return View(bookings);
}
```

```
// — Mark Complete (Teacher action) —————
[HttpPost]
[ValidateAntiForgeryToken]
```

```

public async Task<ActionResult> Complete(int id)
{
    var userId = HttpContext.Session.GetInt32("UserId");
    if (userId == null)
        return RedirectToAction("Login", "Account");

    var booking = await _context.Bookings
        .Include(b => b.Skill)
        .FirstOrDefaultAsync(b => b.Id == id);

    if (booking == null)
        return NotFound();

    // Only the skill owner (teacher) can mark complete
    if (booking.Skill!.UserId != userId)
    {
        TempData["Error"] = "Only the teacher of this skill can mark it complete.";
        return RedirectToAction("TeachingSessions");
    }

    if (booking.Status != "Pending")
    {
        TempData["Error"] = "Only Pending bookings can be marked as complete.";
        return RedirectToAction("TeachingSessions");
    }

    // Mark as complete
    booking.Status = "Completed";

    // Reward teacher with points
    var teacherWallet = await _context.Wallets
        .FirstOrDefaultAsync(w => w.UserId == userId);

    if (teacherWallet != null)
        teacherWallet.Points += booking.Skill.PointCost;

    await _context.SaveChangesAsync();

    TempData["Success"] = $"Session marked complete! You earned {booking.Skill.PointCost} points.";
    return RedirectToAction("TeachingSessions");
}

```

// — Create Booking (Learner books a skill) —————

```
public async Task<ActionResult> Create(int skillId)
{
    var userId = HttpContext.Session.GetInt32("UserId");
    if (userId == null)
        return RedirectToAction("Login", "Account");

    var skill = await _context.Skills
        .Include(s => s.User)
        .FirstOrDefaultAsync(s => s.Id == skillId);

    if (skill == null)
        return NotFound();

    var wallet = await _context.Wallets
        .FirstOrDefaultAsync(w => w.UserId == userId);

    ViewBag.Skill = skill;
    ViewBag.WalletPoints = wallet?.Points ?? 0;

    return View();
}
```

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<ActionResult> Create(int skillId, string? note)
{
    var userId = HttpContext.Session.GetInt32("UserId");
    if (userId == null)
        return RedirectToAction("Login", "Account");

    var skill = await _context.Skills.FindAsync(skillId);
    if (skill == null)
        return NotFound();

    var wallet = await _context.Wallets
        .FirstOrDefaultAsync(w => w.UserId == userId);

    if (wallet == null || wallet.Points < skill.PointCost)
    {
        TempData["Error"] = "Not enough points to book this skill.";
        return RedirectToAction("Details", "Skill", new { id = skillId });
    }
}
```

```

}

wallet.Points -= skill.PointCost;

var booking = new Booking
{
    SkillId = skillId,
    UserId = userId.Value,
    BookedAt = DateTime.Now,
    Status = "Pending"
};

_context.Bookings.Add(booking);
await _context.SaveChangesAsync();

TempData["Success"] = "Booking created! " + skill.PointCost + " points deducted.";
return RedirectToAction("Index");
}

```

// — Cancel Booking (Learner cancels, points refunded) —————

```

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<ActionResult> Cancel(int id)
{
    var userId = HttpContext.Session.GetInt32("UserId");
    if (userId == null)
        return RedirectToAction("Login", "Account");

    var booking = await _context.Bookings
        .Include(b => b.Skill)
        .FirstOrDefaultAsync(b => b.Id == id);

    if (booking == null)
        return NotFound();

    if (booking.UserId != userId)
    {
        TempData["Error"] = "You can only cancel your own bookings.";
        return RedirectToAction("Index");
    }
}

```

```

var wallet = await _context.Wallets

```

```

        .FirstOrDefaultAsync(w => w.UserId == userId);

    if (wallet != null && booking.Skill != null)
        wallet.Points += booking.Skill.PointCost;

    booking.Status = "Cancelled";
    await _context.SaveChangesAsync();

    TempData["Success"] = "Booking cancelled. Points refunded.";
    return RedirectToAction("Index");
}
}
}

```

HomeController.cs

```

using System.Diagnostics;
using Microsoft.AspNetCore.Mvc;
using SmartSkillExchange.Models;

```

namespace SmartSkillExchange.Controllers

```

{
    public class HomeController : Controller
    {
        private readonly ILogger<HomeController> _logger;

        public HomeController(ILogger<HomeController> logger)
        {
            _logger = logger;
        }

        public IActionResult Index()
        {
            return View();
        }

        public IActionResult Privacy()
        {
            return View();
        }
    }
}

```

```

[ResponseCache(Duration = 0, Location = ResponseCacheLocation.None, NoStore = true)]
public IActionResult Error()

```

```

    {
        return View(new ErrorViewModel { RequestId = Activity.Current?.Id ?? HttpContext.TraceIdentifier });
    }
}

```

SkillController.cs

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using SmartSkillExchange.Data;
using SmartSkillExchange.Models;

```

```

namespace SmartSkillExchange.Controllers

```

```

{
    public class SkillController : Controller
    {
        private readonly ApplicationDbContext _context;

        public SkillController(ApplicationDbContext context)
        {
            _context = context;
        }

        public async Task<ActionResult> Index(string? keyword, string? category)
        {
            IQueryable<Skill> query = _context.Skills
                .Include(s => s.User);

            if (!string.IsNullOrEmpty(keyword))
                query = query.Where(s => s.Title.Contains(keyword));

            if (!string.IsNullOrEmpty(category))
                query = query.Where(s => s.Category == category);

            query = query.OrderBy(s => s.Title);

            ViewBag.Keyword = keyword;
            ViewBag.Category = category;
        }
    }
}

```

```
var skills = await query.ToListAsync();
return View(skills);
}
```

```
public async Task<IActionResult> Details(int id)
{
    var skill = await _context.Skills
        .Include(s => s.User)
        .Include(s => s.Reviews)
        .FirstOrDefaultAsync(s => s.Id == id);

    if (skill == null)
        return NotFound();

    return View(skill);
}
```

```
public IActionResult Create()
{
    var userId = HttpContext.Session.GetInt32("UserId");
    if (userId == null)
    {
        TempData["Error"] = "Please login first.";
        return RedirectToAction("Login", "Account");
    }
    return View();
}
```

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create(Skill model)
{
    var userId = HttpContext.Session.GetInt32("UserId");
    if (userId == null)
        return RedirectToAction("Login", "Account");

    ModelState.Remove("User");
}
```



```

ModelState.Remove("Bookings");
ModelState.Remove("Reviews");

if (!ModelState.IsValid)
    return View(model);

model.UserId = userId.Value;
_context.Skills.Add(model);
await _context.SaveChangesAsync();

TempData["Success"] = "Skill added successfully!";
return RedirectToAction("Index");
}

```

```

public async Task<IActionResult> Edit(int id)
{
    var userId = HttpContext.Session.GetInt32("UserId");
    if (userId == null)
        return RedirectToAction("Login", "Account");

    var skill = await _context.Skills.FindAsync(id);

    if (skill == null)
        return NotFound();

    if (skill.UserId != userId)
    {
        TempData["Error"] = "You can only edit your own skills.";
        return RedirectToAction("Index");
    }

    return View(skill);
}

```

```

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int id, Skill model)
{
    var userId = HttpContext.Session.GetInt32("UserId");
    if (userId == null)

```

```

        return RedirectToAction("Login", "Account");

ModelState.Remove("User");
ModelState.Remove("Bookings");
ModelState.Remove("Reviews");

if (!ModelState.IsValid)
    return View(model);

var skill = await _context.Skills.FindAsync(id);

if (skill == null)
    return NotFound();

// Update fields
skill.Title = model.Title;
skill.Description = model.Description;
skill.Category = model.Category;
skill.PointCost = model.PointCost;

await _context.SaveChangesAsync();

TempData["Success"] = "Skill updated successfully!";
return RedirectToAction("Index");
}

```

```

public async Task<IActionResult> Delete(int id)
{
    var userId = HttpContext.Session.GetInt32("UserId");
    if (userId == null)
        return RedirectToAction("Login", "Account");

    var skill = await _context.Skills
        .Include(s => s.User)
        .FirstOrDefaultAsync(s => s.Id == id);

    if (skill == null)
        return NotFound();

    if (skill.UserId != userId)
    {

```

```

        TempData["Error"] = "You can only delete your own skills.";
        return RedirectToAction("Index");
    }

```

```

    return View(skill);
}

```

```

[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var skill = await _context.Skills.FindAsync(id);

    if (skill == null)
        return NotFound();

    _context.Skills.Remove(skill);
    await _context.SaveChangesAsync();

    TempData["Success"] = "Skill deleted successfully.";
    return RedirectToAction("Index");
}
}
}

```

Account/Login.cshtml

```

@{
    ViewData["Title"] = "Login";
}

```

```

<div class="row justify-content-center">
    <div class="col-md-4">
        <h2 class="mb-4 text-center">Login</h2>

        <form asp-action="Login" method="post">
            @Html.AntiForgeryToken()
            <div asp-validation-summary="All" class="alert alert-danger"></div>

            <div class="mb-3">
                <label class="form-label">Email</label>
                <input type="email" name="email" class="form-control"

```

```

        placeholder="you@example.com" required />
</div>

<div class="mb-3">
    <label class="form-label">Password</label>
    <input type="password" name="password" class="form-control"
        placeholder="Password" required />
</div>

    <button type="submit" class="btn btn-primary w-100">Login</button>
</form>

<p class="text-center mt-3">
    No account?
    <a asp-action="Register">Register here</a>
</p>
</div>
</div>

```

Account/register.cshtml

```

@model SmartSkillExchange.Models.User
@{
    ViewData["Title"] = "Register";
}

```

```

<div class="row justify-content-center">
    <div class="col-md-5">
        <h2 class="mb-4 text-center">Create Account</h2>

        <form asp-action="Register" method="post">
            @Html.AntiForgeryToken()
            <div asp-validation-summary="ModelOnly" class="alert alert-danger"></div>

            <div class="mb-3">
                <label asp-for="Name" class="form-label"></label>
                <input asp-for="Name" class="form-control" placeholder="Your full name" />
                <span asp-validation-for="Name" class="text-danger"></span>
            </div>

```

```
<div class="mb-3">
  <label asp-for="Email" class="form-label"></label>
  <input asp-for="Email" class="form-control" placeholder="you@example.com" />
  <span asp-validation-for="Email" class="text-danger"></span>
</div>
```

```
<div class="mb-3">
  <label asp-for="Password" class="form-label"></label>
  <input asp-for="Password" type="password" class="form-control" placeholder="Password" />
  <span asp-validation-for="Password" class="text-danger"></span>
</div>
```

```
<button type="submit" class="btn btn-primary w-100">Register</button>
</form>
```

```
<p class="text-center mt-3">
  Already have an account?
  <a asp-action="Login">Login here</a>
</p>
</div>
</div>
```

```
@section Scripts {
  <script src="https://cdn.jsdelivr.net/npm/jquery@3.7.1/dist/jquery.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/jquery-validation@1.19.5/dist/jquery.validate.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/jquery-validation-unobtrusive@4.0.0/dist/jquery.validate.unobtrusive.min.js"></script>
}
Admin/AllBooking.cshtml
```

```
@model List<SmartSkillExchange.Models.Booking>
@{
  ViewData["Title"] = "All Bookings";
}
```

```
<h2 class="mb-4">All Bookings</h2>
```

```
<table class="table table-bordered table-hover">
  <thead class="table-dark">
    <tr>
```

```

        <th>Id</th>
        <th>User</th>
        <th>Skill</th>
        <th>Status</th>
        <th>Booked On</th>
    </tr>
</thead>
<tbody>
    @foreach (var item in Model)
    {
        <tr>
            <td>@item.Id</td>
            <td>@item.User?.Name</td>
            <td>@item.Skill?.Title</td>
            <td>
                <span class="badge @(item.Status == "Pending" ? "bg-warning text-dark" :
                    item.Status == "Cancelled" ? "bg-danger" : "bg-success")">
                    @item.Status
                </span>
            </td>
            <td>@item.BookedAt.ToString("dd MMM yyyy")</td>
        </tr>
    }
</tbody>
</table>

```

```

<a asp-action="Index" class="btn btn-secondary mt-2">Back to Admin</a>
Admin.Index.cshtml

```

```

@model List<SmartSkillExchange.Models.User>
@{
    ViewData["Title"] = "Admin Panel";
}

```

```

<h2 class="mb-4">Admin Panel</h2>

```

```

<div class="row g-3 mb-4">
    <div class="col-md-3">
        <div class="card text-center shadow-sm">
            <div class="card-body">
                <h3 class="text-primary">@ViewBag.TotalUsers</h3>
                <p class="mb-0">Total Users</p>
            </div>
        </div>
    </div>
</div>

```

```

        </div>
    </div>
</div>
<div class="col-md-3">
    <div class="card text-center shadow-sm">
        <div class="card-body">
            <h3 class="text-success">@ViewBag.TotalSkills</h3>
            <p class="mb-0">Total Skills</p>
        </div>
    </div>
</div>
</div>
<div class="col-md-3">
    <div class="card text-center shadow-sm">
        <div class="card-body">
            <h3 class="text-warning">@ViewBag.TotalBookings</h3>
            <p class="mb-0">Total Bookings</p>
        </div>
    </div>
</div>
</div>
</div>

```

```

<h4>All Users</h4>

```

```

<table class="table table-bordered table-hover">
    <thead class="table-dark">
        <tr>
            <th>Id</th>
            <th>Name</th>
            <th>Email</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var user in Model)
        {
            <tr>
                <td>@user.Id</td>
                <td>@user.Name</td>
                <td>@user.Email</td>
            </tr>
        }
    </tbody>
</table>

```

```
<a asp-action="AllBookings" class="btn btn-outline-primary mt-2">View All Bookings</a>
```

Booking/Create.cshtml

```
@{
    ViewData["Title"] = "Book Skill";
    var skill = ViewBag.Skill as SmartSkillExchange.Models.Skill;
    int walletPoints = ViewBag.WalletPoints;
}

<h2 class="mb-4">Book a Session</h2>

<div class="card col-md-5">
    <div class="card-body">
        <h5 class="card-title">@skill?.Title</h5>
        <p class="text-muted">@skill?.Description</p>
        <p><strong>Category:</strong> @skill?.Category</p>
        <p>
            <strong>Cost:</strong>
            <span class="badge bg-warning text-dark">@skill?.PointCost pts</span>
        </p>
        <p>
            <strong>Your Balance:</strong>
            <span class="badge @(walletPoints >= (skill?.PointCost ?? 0) ? "bg-success" : "bg-danger")">
                @walletPoints pts
            </span>
        </p>

        @if (walletPoints < (skill?.PointCost ?? 0))
        {
            <div class="alert alert-danger">
                Not enough points to book this skill.
            </div>
        }
        else
        {
            <form asp-action="Create" method="post">
                @Html.AntiForgeryToken()
                <input type="hidden" name="skillId" value="@skill?.Id" />

                <div class="mb-3">
                    <label class="form-label">Note (optional)</label>
                </div>
            </form>
        }
    </div>
</div>
```



```
<textarea name="note" class="form-control" rows="2"
  placeholder="Any message for the teacher..."></textarea>
</div>
```

```
<button type="submit" class="btn btn-success w-100">
  Confirm Booking (@skill?.PointCost pts)
</button>
</form>
}
```

```
<a asp-controller="Skill" asp-action="Index" class="btn btn-secondary w-100 mt-2">
  Back to Skills
</a>
</div>
</div>
```

Booking/Index.cshtml

```
@model List<Booking>
```

```
@{
  ViewData["Title"] = "My Bookings";
}
```

```
<div class="d-flex justify-content-between align-items-center mb-4">
  <h2 class="mb-0">My Bookings <small class="text-muted fs-6">(as Learner)</small></h2>
  <a asp-controller="Booking" asp-action="TeachingSessions" class="btn btn-success">
    🎓 View Teaching Sessions
  </a>
</div>
```

```
@if (!Model.Any())
{
  <p class="text-muted">
    You have no bookings yet.
    <a asp-controller="Skill" asp-action="Index">Browse skills</a>
  </p>
}
else
{
  <table class="table table-bordered table-hover">
    <thead class="table-primary">
      <tr>
```

```

        <th>Skill</th>
        <th>Category</th>
        <th>Points Spent</th>
        <th>Status</th>
        <th>Booked On</th>
        <th>Action</th>
    </tr>
</thead>
<tbody>
    @foreach (var item in Model)
    {
        <tr>
            <td>@item.Skill?.Title</td>
            <td>@item.Skill?.Category</td>
            <td>@item.Skill?.PointCost pts</td>
            <td>
                <span class="badge @(item.Status == "Pending" ? "bg-warning text-dark" :
                    item.Status == "Cancelled" ? "bg-danger" : "bg-success")">
                    @item.Status
                </span>
            </td>
            <td>@item.BookedAt.ToString("dd MMM yyyy")</td>
            <td>
                @if (item.Status == "Pending")
                {
                    <form asp-action="Cancel" asp-route-id="@item.Id" method="post">
                        @Html.AntiForgeryToken()
                        <button type="submit" class="btn btn-sm btn-danger"
                            onclick="return confirm('Cancel this booking?')">
                            Cancel
                        </button>
                    </form>
                }
            </td>
        </tr>
    }
</tbody>
</table>
}

```

Booking/TeachingSessions.cshtml
 @model List<Booking>

```
@{
    ViewData["Title"] = "My Teaching Sessions";
}
```

```
<h2 class="mb-4">My Teaching Sessions</h2>
```

```
<p class="text-muted">These are bookings made by other users for <strong>your skills</strong>. Mark
them complete after delivering the session to earn points.</p>
```

```
@if (!Model.Any())
{
    <div class="alert alert-info">
        No one has booked your skills yet.
        <a asp-controller="Skill" asp-action="Index">Browse skills</a>
    </div>
}
```

```
else
{
```

```
    <table class="table table-bordered table-hover">
        <thead class="table-success">
            <tr>
                <th>Skill</th>
                <th>Learner</th>
                <th>Points</th>
                <th>Booked On</th>
                <th>Status</th>
                <th>Action</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var item in Model)
            {
                <tr>
                    <td>@item.Skill?.Title</td>
                    <td>@item.User?.Name</td>
                    <td>@item.Skill?.PointCost pts</td>
                    <td>@item.BookedAt.ToString("dd MMM yyyy")</td>
                    <td>
                        <span class="badge @(item.Status == "Pending" ? "bg-warning text-dark" :
                            item.Status == "Completed" ? "bg-success" : "bg-danger")">
                            @item.Status
                        </span>
                    </td>
                </tr>
            }
        </tbody>
    </table>
```

```

<td>
  @if (item.Status == "Pending")
  {
    <form asp-action="Complete" asp-route-id="@item.Id" method="post">
      @Html.AntiForgeryToken()
      <button type="submit" class="btn btn-sm btn-success"
        onclick="return confirm('Mark this session as Completed? You will earn
@(item.Skill?.PointCost) points.')">
         Mark Complete
      </button>
    </form>
  }
  else
  {
    <span class="text-muted">—</span>
  }
</td>
</tr>
}
</tbody>
</table>
}

```

Skill/Create.cshtml

```

@model Skill
@{
  ViewData["Title"] = "Add Skill";
}

```

```
<h2 class="mb-4">Add New Skill</h2>
```

```

<div class="col-md-6">
  <form asp-action="Create" method="post">
    @Html.AntiForgeryToken()
    <div asp-validation-summary="ModelOnly" class="alert alert-danger"></div>

    <div class="mb-3">
      <label asp-for="Title" class="form-label"></label>
      <input asp-for="Title" class="form-control" placeholder="e.g. Python Programming" />
      <span asp-validation-for="Title" class="text-danger"></span>
    </div>

```

```

<div class="mb-3">
  <label asp-for="Description" class="form-label"></label>
  <textarea asp-for="Description" class="form-control" rows="3"
    placeholder="Describe what you will teach..."></textarea>
  <span asp-validation-for="Description" class="text-danger"></span>
</div>

```

```

<div class="mb-3">
  <label asp-for="Category" class="form-label"></label>
  <input asp-for="Category" class="form-control" placeholder="e.g. Technology" />
  <span asp-validation-for="Category" class="text-danger"></span>
</div>

```

```

<div class="mb-3">
  <label asp-for="PointCost" class="form-label"></label>
  <input asp-for="PointCost" type="number" class="form-control" placeholder="e.g. 30" />
  <span asp-validation-for="PointCost" class="text-danger"></span>
</div>

```

```

<div class="d-flex gap-2">
  <button type="submit" class="btn btn-primary">Save Skill</button>
  <a asp-action="Index" class="btn btn-secondary">Cancel</a>
</div>

```

```

</form>
</div>

```

```

@section Scripts {
  <script src="https://cdn.jsdelivr.net/npm/jquery@3.7.1/dist/jquery.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/jquery-validation@1.19.5/dist/jquery.validate.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/jquery-validation-unobtrusive@4.0.0/dist/jquery.validate.unobtrusive.min.js"></script>
}
Skill/delete.cshtml

```

```

@model Skill
@{
  ViewData["Title"] = "Delete Skill";
}

```

```

<h2 class="mb-4">Delete Skill</h2>

```

```
<div class="alert alert-danger">
  Are you sure you want to delete <strong>@Model.Title</strong>?
</div>
```

```
<table class="table table-bordered w-auto mb-4">
  <tr><th>Title</th><td>@Model.Title</td></tr>
  <tr><th>Category</th><td>@Model.Category</td></tr>
  <tr><th>Point Cost</th><td>@Model.PointCost pts</td></tr>
</table>
```

```
<form asp-action="Delete" asp-route-id="@Model.Id" method="post">
  @Html.AntiForgeryToken()
  <button type="submit" class="btn btn-danger">Yes, Delete</button>
  <a asp-action="Index" class="btn btn-secondary ms-2">Cancel</a>
</form>
```

Skill/details.cshtml

```
@model Skill
@{
  ViewData["Title"] = Model.Title;
}
```

```
<h2>@Model.Title</h2>
<p class="text-muted">@Model.Description</p>
```

```
<table class="table table-bordered w-auto">
  <tr>
    <th>Category</th>
    <td><span class="badge bg-primary">@Model.Category</span></td>
  </tr>
  <tr>
    <th>Point Cost</th>
    <td><span class="badge bg-warning text-dark">@Model.PointCost pts</span></td>
  </tr>
  <tr>
    <th>Offered By</th>
    <td>@Model.User?.Name</td>
  </tr>
</table>
```

```

<h5 class="mt-4">Reviews (@Model.Reviews.Count)</h5>
@if (!Model.Reviews.Any())
{
    <p class="text-muted">No reviews yet.</p>
}
else
{
    @foreach (var r in Model.Reviews)
    {
        <div class="card mb-2">
            <div class="card-body">
                <p class="mb-1">@r.Comment</p>
                <span class="badge bg-warning text-dark">Rating: @r.Rating / 5</span>
            </div>
        </div>
    }
}

```

```

<div class="mt-3 d-flex gap-2">
    <a asp-action="Index" class="btn btn-secondary">Back to List</a>

```

```

@if (Context.Session.GetInt32("UserId") != null &&
    Context.Session.GetInt32("UserId") != Model.UserId)
{
    <a asp-controller="Booking" asp-action="Create"
        asp-route-skillId="@Model.Id"
        class="btn btn-success">Book This Skill</a>
}
</div>

```

Skill/index.cshtml

```
@model List<Skill>
```

```

@{
    ViewData["Title"] = "All Skills";
}

```

```
<h2 class="mb-4">Browse Skills</h2>
```

```

<form asp-action="Index" method="get" class="row g-2 mb-4">
    <div class="col-md-4">
        <input type="text" name="keyword" value="@ViewBag.Keyword"
            class="form-control" placeholder="Search by title..." />
    </div>

```

```

<div class="col-md-3">
    <input type="text" name="category" value="@ViewBag.Category"
        class="form-control" placeholder="Category..." />
</div>
<div class="col-md-2">
    <button type="submit" class="btn btn-primary w-100">Search</button>
</div>
<div class="col-md-2">
    <a asp-action="Index" class="btn btn-secondary w-100">Clear</a>
</div>
</form>

```

```

@if (!Model.Any())
{
    <p class="text-muted">No skills found.</p>
}
else
{
    <div class="row row-cols-1 row-cols-md-3 g-4">
        @foreach (var item in Model)
        {
            <div class="col">
                <div class="card h-100 shadow-sm">
                    <div class="card-body">
                        <h5 class="card-title">@item.Title</h5>
                        <p class="card-text text-muted">@item.Description</p>
                        <span class="badge bg-primary">@item.Category</span>
                        <span class="badge bg-warning text-dark ms-1">@item.PointCost pts</span>
                        <p class="mt-2 mb-0 small text-muted">By: @item.User?.Name</p>
                    </div>
                    <div class="card-footer d-flex gap-2">
                        <a asp-action="Details" asp-route-id="@item.Id"
                            class="btn btn-sm btn-outline-primary">Details</a>

                        @if (Context.Session.GetInt32("UserId") == item.UserId)
                        {
                            <a asp-action="Edit" asp-route-id="@item.Id"
                                class="btn btn-sm btn-outline-secondary">Edit</a>
                            <a asp-action="Delete" asp-route-id="@item.Id"
                                class="btn btn-sm btn-outline-danger">Delete</a>
                        }
                    </div>
                </div>
            </div>
        }
    </div>

```



```

        @if (Context.Session.GetInt32("UserId") != null &&
            Context.Session.GetInt32("UserId") != item.UserId)
        {
            <a asp-controller="Booking" asp-action="Create"
              asp-route-skillId="@item.Id"
              class="btn btn-sm btn-success">Book</a>
        }
    </div>
</div>
</div>
}
</div>
}

```

Skill/Edit.cshtml

```
@model Skill
```

```

@{
    ViewData["Title"] = "Edit Skill";
}

```

```
<h2 class="mb-4">Edit Skill</h2>
```

```
<div class="col-md-6">
```

```

    <form asp-action="Edit" asp-route-id="@Model.Id" method="post">
        @Html.AntiForgeryToken()
        <input type="hidden" asp-for="Id" />
        <input type="hidden" asp-for="UserId" />
        <div asp-validation-summary="ModelOnly" class="alert alert-danger"></div>

```

```
<div class="mb-3">
```

```

    <label asp-for="Title" class="form-label"></label>
    <input asp-for="Title" class="form-control" />
    <span asp-validation-for="Title" class="text-danger"></span>
</div>

```

```
<div class="mb-3">
```

```

    <label asp-for="Description" class="form-label"></label>
    <textarea asp-for="Description" class="form-control" rows="3"></textarea>
    <span asp-validation-for="Description" class="text-danger"></span>
</div>

```

```

<div class="mb-3">
  <label asp-for="Category" class="form-label"></label>
  <input asp-for="Category" class="form-control" />
  <span asp-validation-for="Category" class="text-danger"></span>
</div>

<div class="mb-3">
  <label asp-for="PointCost" class="form-label"></label>
  <input asp-for="PointCost" type="number" class="form-control" />
  <span asp-validation-for="PointCost" class="text-danger"></span>
</div>

<div class="d-flex gap-2">
  <button type="submit" class="btn btn-primary">Update Skill</button>
  <a asp-action="Index" class="btn btn-secondary">Cancel</a>
</div>
</form>
</div>

```

```

@section Scripts {
  <script src="https://cdn.jsdelivr.net/npm/jquery@3.7.1/dist/jquery.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/jquery-
validation@1.19.5/dist/jquery.validate.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/jquery-validation-
unobtrusive@4.0.0/dist/jquery.validate.unobtrusive.min.js"></script>
}

```

```

Appsettings.json
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=TUSHAR-
SINGHAL;Database=SmartSkillExchangeDb;Trusted_Connection=True;MultipleActiveResultSets=true;Tr
ustServerCertificate=True"

  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}

```

```
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
```

```
using SmartSkillExchange.Data;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services.AddControllersWithViews();
```

```
builder.Services.AddDbContext<AppDbContext>(options =>  
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

```
builder.Services.AddSession(options =>  
{  
    options.IdleTimeout = TimeSpan.FromHours(1);  
    options.Cookie.HttpOnly = true;  
    options.Cookie.IsEssential = true;  
});
```

```
builder.Services.AddMemoryCache();
```

```
var app = builder.Build();
```

```
if (!app.Environment.IsDevelopment())  
{  
    app.UseExceptionHandler("/Home/Error");  
    app.UseHsts();  
}
```

```
app.UseHttpsRedirection();  
app.UseStaticFiles();  
app.UseRouting();
```

```
app.UseSession();
```

```
app.UseAuthorization();
```

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

```
app.MapControllerRoute(  
    name: "skill_details",  
    pattern: "Skill/Details/{id:int}",  
    defaults: new { controller = "Skill", action = "Details" });  
  
app.Run();
```

Question 2:- order api

Order.cs

```
using System.ComponentModel.DataAnnotations;
```

```
namespace OrderApi.Models
```

```
{  
    public class Order  
    {  
        public int Id { get; set; }  
  
        [Required]  
        public string CustomerName { get; set; } = string.Empty;  
  
        public double TotalAmount { get; set; }  
  
        public OrderStatus Status { get; set; }  
    }  
  
    public enum OrderStatus  
    {  
        Pending,  
        Processing,  
        Completed,  
        Cancelled,  
        HighValueOrder  
    }  
}
```

```
public enum OrderStatus
```

```
{  
    Pending,  
    Processing,  
    Completed,  
    Cancelled,  
    HighValueOrder  
}  
}
```

OrderDto.cs

```
namespace OrderApi.DTOs
```

```
{  
    public class OrderDto  
    {  
        public string CustomerName { get; set; } = string.Empty;  
  
        public double TotalAmount { get; set; }  
    }  
}
```

```
}  
}
```

MappingProfiles.cs

using AutoMapper;

using OrderApi.DTOs;

using OrderApi.Models;

namespace OrderApi.Mapping

```
{  
    public class MappingProfile : Profile
```

```
{  
    public MappingProfile()  
    {
```

```
        CreateMap<OrderDto, Order>()  
            .ForMember(dest => dest.Status, opt => opt.Ignore());
```

```
        CreateMap<Order, OrderDto>();
```

```
    }  
}
```

IOrderRepository

using OrderApi.Models;

namespace OrderApi.Repository

```
{  
    public interface IOrderRepository
```

```
{  
    public Task<IEnumerable<Order>> GetAllOrders();
```

```
    public Task<Order> GetOrderById(int id);
```

```
    public Task<Order> CreateOrder(Order order);
```

```
    public Task<Order> UpdateStatus(int id, string status);
```

```
    public Task<bool> DeleteOrder(int id);
```

```
    }  
}
```

```

OrderRepository
using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using OrderApi.Data;
using OrderApi.Models;

namespace OrderApi.Repository
{
    public class OrderRepository : IOrderRepository
    {

        private readonly AppDbContext _context;

        public OrderRepository(AppDbContext context)
        {
            _context = context;
        }

        public async Task<IEnumerable<Order>> GetAllOrders()
        {
            return await _context.Orders.ToListAsync();
        }

        public async Task<Order?> GetOrderById(int id)
        {
            return await _context.Orders.FindAsync(id);
        }

        public async Task<Order> CreateOrder(Order order)
        {
            var TotalAmount = order.TotalAmount;

            if (TotalAmount > 50000)
            {
                order.Status = OrderStatus.HighValueOrder;
            }
            else
            {
                order.Status = OrderStatus.Pending;
            }
        }
    }
}

```

```

    }
    _context.Orders.Add(order);

    await _context.SaveChangesAsync();

    return order;

}

public async Task<Order?> UpdateStatus(int id, string status)
{
    var order = await _context.Orders.FindAsync(id);
    if (order != null)
    {
        if (Enum.TryParse<OrderStatus>(status, true, out var newStatus))
        {
            order.Status = newStatus;
            await _context.SaveChangesAsync();
            return order;
        }
    }
    return null;
}

public async Task<bool> DeleteOrder(int id)
{
    var order = await _context.Orders.FindAsync(id);
    if (order != null)
    {
        _context.Orders.Remove(order);
        await _context.SaveChangesAsync();
        return true;
    }
    return false;
}
}
}

```

OrderBackgroundService.cs

```

using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;

```

```

namespace OrderApi.Services
{
    public class OrderBackgroundService : BackgroundService
    {
        private readonly ILogger<OrderBackgroundService> _logger;

        public OrderBackgroundService(ILogger<OrderBackgroundService> logger)
        {
            _logger = logger;
        }

        protected override async Task ExecuteAsync(CancellationToken stoppingToken)
        {
            while (!stoppingToken.IsCancellationRequested)
            {
                _logger.LogInformation("Checking pending orders...");

                await Task.Delay(10000, stoppingToken);
            }
        }
    }
}

```

```

OrderController.cs
using AutoMapper;
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Tokens;
using OrderApi.DTOs;
using OrderApi.Models;
using OrderApi.Repository;

namespace OrderApi.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class OrderController : Controller
    {
        private readonly IOrderRepository _orderRepository;

        private readonly IMapper _mapper;
    }
}

```



```
public OrderController(IOrderRepository orderRepository, IMapper mapper)
{
    _orderRepository = orderRepository;
    _mapper = mapper;
}
```

```
[HttpGet]
public async Task<ActionResult> GetAll()
{
    var orders = await _orderRepository.GetAllOrders();

    return Ok(orders);
}
```

```
[HttpGet("{id}")]
public async Task<ActionResult> GetById(int id)
{
    var order = await _orderRepository.GetOrderById(id);

    if (order == null)
    {
        return NotFound();
    }

    return Ok(order);
}
```

```
[HttpPost]
public async Task<ActionResult> Create(OrderDto orderDto)
{
    if(string.IsNullOrEmpty(orderDto.CustomerName))
    {
        return BadRequest("Customer name is required");
    }

    if (orderDto.TotalAmount <= 0)
    {
        return BadRequest("Amount must be greater than 0");
    }
}
```

```

var order = _mapper.Map<Order>(orderDto);

order.Status = order.TotalAmount > 100 ? OrderStatus.HighValueOrder : OrderStatus.Pending;

var created = await _orderRepository.CreateOrder(order);

var CreatedAtActionResult = CreatedAtAction(nameof(GetById), new { id = created.Id }, created);

return CreatedAtActionResult;
}

[HttpPut("{id}")]
public async Task<IActionResult> UpdateStatus(int id, [FromBody] string status)
{
    // Validate allowed values only
    if (!Enum.TryParse<OrderStatus>(status, true, out var parsedStatus))
        return BadRequest("Invalid status");

    if (parsedStatus != OrderStatus.Pending &&
        parsedStatus != OrderStatus.Completed &&
        parsedStatus != OrderStatus.Cancelled)
    {
        return BadRequest("Only Pending, Completed, Cancelled allowed");
    }

    var updated = await _orderRepository.UpdateStatus(id, parsedStatus.ToString());

    if (updated == null)
        return NotFound();

    return Ok(updated);
}

[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{
    var success = await _orderRepository.DeleteOrder(id);

    if (!success)
        return NotFound();

    return NoContent();
}

```

```
}
```

```
}
```

```
}
```

```
ExceptionMiddleware.cs
```

```
using System.Net;
```

```
using System.Text.Json;
```

```
namespace OrderApi.Middleware
```

```
{
```

```
    public class ExceptionMiddleware
```

```
    {
```

```
        private readonly RequestDelegate _next;
```

```
        private readonly ILogger<ExceptionMiddleware> _logger;
```

```
        public ExceptionMiddleware(RequestDelegate next, ILogger<ExceptionMiddleware> logger)
```

```
        {
```

```
            _next = next;
```

```
            _logger = logger;
```

```
        }
```

```
        public async Task InvokeAsync(HttpContext context)
```

```
        {
```

```
            try
```

```
            {
```

```
                await _next(context);
```

```
            }
```

```
            catch (Exception ex)
```

```
            {
```

```
                _logger.LogError(ex, "Unhandled exception occurred");
```

```
                context.Response.StatusCode = (int)HttpStatusCode.InternalServerError;
```

```
                context.Response.ContentType = "application/json";
```

```
                var response = new
```

```
                {
```

```
                    error = ex.Message,
```

```
                    stackTrace = ex.StackTrace
```

```

};

var json = JsonSerializer.Serialize(response);

await context.Response.WriteAsync(json);
}
}
}
}
}

```

Program.cs

```

using Microsoft.EntityFrameworkCore;
using OrderApi.Data;
using OrderApi.Middleware;
using OrderApi.Services;
using OrderApi.Repository;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddScoped<IOrderRepository, OrderRepository>();

builder.Services.AddAutoMapper(typeof(Program));

builder.Services.AddHostedService<OrderBackgroundService>();
builder.Services.AddControllers()
    .AddJsonOptions(options =>
    {
        options.JsonSerializerOptions.Converters.Add(
            new System.Text.Json.Serialization.JsonStringEnumConverter()
        );
    });
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())

```

```

{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseMiddleware<ExceptionMiddleware>();

app.UseHttpsRedirection();

app.UseRouting();

app.UseAuthorization();

app.MapControllers();

app.Run();
AppSettings.json

{
    "ConnectionStrings": {
        "DefaultConnection": "Server=TUSHAR-
SINGHAL;Database=OrderDb;Trusted_Connection=True;TrustServerCertificate=True;"
    },
    "Logging": {
        "LogLevel": {
            "Default": "Information",
            "Microsoft.AspNetCore": "Warning"
        }
    },
    "AllowedHosts": "*"
}

```

Question 1: Area & Perimeter of Rectangle

```
using System;
```

```
class Shape
```

```

{

    public Shape()
    {

```

```
}  
}
```

```
class Rectangle : Shape
```

```
{  
    private int length;  
    private int width;
```

```
  
    public Rectangle(int length, int width) : base()  
    {  
        this.length = length;  
        this.width = width;  
    }
```

```
  
    public int CalculateArea()  
    {  
        return length * width;  
    }
```

```
  
    public int CalculatePerimeter()  
    {  
        return 2 * (length + width);  
    }  
}
```

```
class Program
```

```
{  
    static void Main()  
    {  
        string[] input = Console.ReadLine().Split();  
  
        int length = int.Parse(input[0]);  
        int width = int.Parse(input[1]);  
  
        Rectangle rect = new Rectangle(length, width);  
  
        Console.WriteLine(rect.CalculateArea());  
        Console.WriteLine(rect.CalculatePerimeter());  
    }  
}
```

✔ Question 2: Supermarket Store (OOP)

```
using System;
using System.Collections.Generic;
using System.Globalization;

// Base Class
class Product
{
    protected string name;
    protected double price;
    protected int quantity;

    public Product(string name, double price, int quantity)
    {
        this.name = name;
        this.price = price;
        this.quantity = quantity;
    }

    public string Name => name;
    public double Price => price;
    public int Quantity => quantity;

    public virtual string GetProductDetails()
    {
        return $"{name} - Price: {price}, Quantity: {quantity}";
    }
}

// Derived Class - Electronics
class Electronics : Product
{
    private int warrantyPeriod;

    public Electronics(string name, double price, int quantity, int warrantyPeriod)
        : base(name, price, quantity)
    {
        this.warrantyPeriod = warrantyPeriod;
    }
}
```

```

public override string GetProductDetails()
{
    return $"{name} - Price: {price}, Quantity: {quantity}, Warranty: {warrantyPeriod} months";
}
}

```

// Derived Class - Clothing

```

class Clothing : Product
{
    private string size;

    public Clothing(string name, double price, int quantity, string size)
        : base(name, price, quantity)
    {
        this.size = size;
    }

    public override string GetProductDetails()
    {
        return $"{name} - Price: {price}, Quantity: {quantity}, Size: {size}";
    }
}

```

// Inventory Class

```

class Inventory
{
    private List<Product> products = new List<Product>();

    public void AddProduct(Product product)
    {
        products.Add(product);
        Console.WriteLine($"Product added to inventory: {product.Name}");
    }

    public void DisplayInventory()
    {
        Console.WriteLine("\nInventory:");
        foreach (var product in products)
        {
            Console.WriteLine(product.GetProductDetails());
        }
    }
}

```



```

public void CalculateTotalValue()
{
    double total = 0;
    foreach (var product in products)
    {
        total += product.Price * product.Quantity;
    }

    Console.WriteLine($"Total value of the inventory: {total:F2}");
}

// Main Program
class Program
{
    static void Main()
    {
        int num = int.Parse(Console.ReadLine());
        Inventory inventory = new Inventory();

        for (int i = 0; i < num; i++)
        {
            string input = Console.ReadLine();
            string[] parts = input.Split(',');

            string type = parts[0].Trim();
            string name = parts[1].Trim();
            double price = double.Parse(parts[2].Trim(), CultureInfo.InvariantCulture);
            int quantity = int.Parse(parts[3].Trim());

            if (type == "Electronics")
            {
                int warranty = int.Parse(parts[4].Trim());
                inventory.AddProduct(new Electronics(name, price, quantity, warranty));
            }
            else if (type == "Clothing")
            {
                string size = parts[4].Trim();
                inventory.AddProduct(new Clothing(name, price, quantity, size));
            }
        }
    }
}

```

```

        inventory.DisplayInventory();
        inventory.CalculateTotalValue();
    }
}

```

Question 3: Virus Resolving

```

using System;
using System.Linq;

```

```

class Program

```

```

{
    static void Main()
    {
        var firstLine = Console.ReadLine().Split().Select(int.Parse).ToArray();
        int N = firstLine[0];
        int X = firstLine[1];
        int Y = firstLine[2];
    }
}

```

```

        var damages = Console.ReadLine().Split().Select(int.Parse).ToArray();
    }
}

```

```

        int result = VirusResolving(N, X, Y);
        Console.WriteLine(result);
    }
}

```

```

static int VirusResolving(int N, int X, int Y)
{
    if (X < 2 * Y)
    {
        int pairs = N / 2;
        int remaining = N % 2;
        return pairs * X + remaining * Y;
    }
    else
    {
        return N * Y;
    }
}
}

```

✔ Question 1: Conditional View Rendering (MVC + Logic)

ScoreModel.cs

```
public class ScoreModel
{
    public string Name { get; set; }
    public int Marks { get; set; }
}
```

ScoreController.cs

```
public class ScoreController : Controller
{
    [HttpGet]
    public IActionResult Index()
    {
        return View();
    }

    [HttpPost]
    public IActionResult Index(ScoreModel model)
    {
        return View(model);
    }
}
```

Index.html

@model ScoreModel

```
<form method="post">
    <input type="text" name="Name" placeholder="Enter Name" required />
    <input type="number" name="Marks" placeholder="Enter Marks" required />
    <button type="submit">Submit</button>
</form>
```

@if (Model != null)

```
{
    if (Model.Marks >= 50)
    {
        <h2 style="color: green;">Pass</h2>
    }
    else
    {
        <h2 style="color: red;">Fail</h2>
    }
}
```

```
}  
}
```

✅ Question 2: Custom Middleware for Request Timing

RequestTimingMiddleware.cs

```
public class RequestTimingMiddleware  
{  
    private readonly RequestDelegate _next;  
  
    public RequestTimingMiddleware(RequestDelegate next)  
    {  
        _next = next;  
    }  
  
    public async Task Invoke(HttpContext context)  
    {  
        var startTime = DateTime.UtcNow;  
  
        await _next(context);  
  
        var duration = DateTime.UtcNow - startTime;  
        var timeTaken = $"{duration.TotalMilliseconds}ms";  
  
        context.Response.Headers["X-Response-Time"] = timeTaken;  
    }  
}
```

Program.cs

```
app.UseMiddleware<RequestTimingMiddleware>();
```

✅ Question 3: LINQ Projection (EF Core Optimization)

Employee.cs

```
public class Employee  
{  
    public int Id { get; set; }  
    public string Name { get; set; }  
    public decimal Salary { get; set; }  
    public string Department { get; set; }  
}
```

Data/AppDbContext.cs

```

using Microsoft.EntityFrameworkCore;
using YourProject.Models;

namespace Optimization.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        public DbSet<Employee> Employees { get; set; }
    }
}

```

Controllers/EmployeesController.cs

```

using Microsoft.AspNetCore.Mvc;
using YourProject.Data;
using System.Linq;

namespace Optimization.Controllers
{
    [ApiController]
    [Route("api/employees")]
    public class EmployeesController : ControllerBase
    {
        private readonly AppDbContext _context;

        public EmployeesController(AppDbContext context)
        {
            _context = context;
        }

        [HttpGet]
        public IActionResult Get()
        {
            var result = _context.Employees
                .Select(e => new
                {
                    e.Name,

```

```

        e.Department
    })
    .ToList();

    return Ok(result);
}
}
}

```

Program.cs

```

using Microsoft.EntityFrameworkCore;
using Optimization.Data;

```

```

var builder = WebApplication.CreateBuilder(args);

```

```

builder.Services.AddControllers();

```

```

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseInMemoryDatabase("EmployeeDb"));

```

```

var app = builder.Build();

```

```

app.UseRouting();
app.UseEndpoints(endpoints =>
{
    endpoints.MapControllers();
});

```

```

using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();

```

```

    db.Employees.AddRange(
        new { Id = 1, Name = "Rahul", Salary = 50000, Department = "IT" },
        new { Id = 2, Name = "Amit", Salary = 60000, Department = "HR" },
        new { Id = 3, Name = "Neha", Salary = 70000, Department = "Finance" }
    ).Select(e => new YourProject.Models.Employee
    {
        Id = e.Id,
        Name = e.Name,
        Salary = e.Salary,
        Department = e.Department
    });
}

```

```

    })
};

db.SaveChanges();
}

app.Run();

```

Question 4: API with Dynamic Filtering (Query Parameters)

Product.cs

```

namespace ApiFil.Models
{
    public class Product
    {
        public int Id { get; set; } // Added for DB realism
        public string Name { get; set; }
        public decimal Price { get; set; }
    }
}

```

Data/AppDbContext.cs

```

using Microsoft.EntityFrameworkCore;
using YourProject.Models;

namespace ApiFil.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        public DbSet<Product> Products { get; set; }
    }
}

```

Controllers/ProductsController.cs

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using YourProject.Data;
using System.Linq;

```

```

namespace ApiFil.Controllers
{
    [ApiController]
    [Route("api/products")]
    public class ProductsController : ControllerBase
    {
        private readonly AppDbContext _context;

        public ProductsController(AppDbContext context)
        {
            _context = context;
        }

        // GET: api/products?minPrice=100&maxPrice=500
        [HttpGet]
        public IActionResult Get(decimal? minPrice, decimal? maxPrice)
        {
            IQueryable<Models.Product> query = _context.Products;

            // Apply filters dynamically
            if (minPrice.HasValue)
            {
                query = query.Where(p => p.Price >= minPrice.Value);
            }

            if (maxPrice.HasValue)
            {
                query = query.Where(p => p.Price <= maxPrice.Value);
            }

            var result = query.ToList();

            return Ok(result);
        }
    }
}

```

Program.cs

```

using Microsoft.EntityFrameworkCore;
using YourProject.Data;

```



```

using ApiFil.Models;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseInMemoryDatabase("ProductDb"));

var app = builder.Build();

app.UseRouting();

app.UseEndpoints(endpoints =>
{
    endpoints.MapControllers();
});

using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();

    db.Products.AddRange(
        new Product { Id = 1, Name = "Laptop", Price = 50000 },
        new Product { Id = 2, Name = "Mouse", Price = 500 },
        new Product { Id = 3, Name = "Keyboard", Price = 1500 },
        new Product { Id = 4, Name = "Monitor", Price = 10000 }
    );

    db.SaveChanges();
}

app.Run();

```

Question 5: Cookie-Based Visit Counter (State Management)

Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```
namespace Cookie.Controllers
```

```
{
    public class HomeController : Controller
    {

```

```

public IActionResult Index()
{
    int visitCount = 1;

    if (Request.Cookies.ContainsKey("VisitCount"))
    {
        visitCount = int.Parse(Request.Cookies["VisitCount"]) + 1;
    }

    Response.Cookies.Append("VisitCount", visitCount.ToString(), new CookieOptions
    {
        Expires = DateTimeOffset.UtcNow.AddDays(7),
        HttpOnly = true,
        IsEssential = true
    });

    ViewBag.VisitCount = visitCount;

    return View();
}
}

```

Views/Home/Index.cshtml

```
<h2>You have visited this page @ViewBag.VisitCount times</h2>
```

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services.AddControllersWithViews();
```

```
var app = builder.Build();
```

```
app.UseStaticFiles();
```

```
app.UseRouting();
```

```
app.UseEndpoints(endpoints =>
```

```
{
    endpoints.MapControllerRoute(
```

```
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}";  
});  
  
app.Run();
```